

go_ordered_planted_fail_first (golden)

Uses `../_shared/affected.json` (members declared in section order `a`, `b`, `d`, `c` — the planted-failing gate `gate_c_planted_fail` is declared **last**) and `../_shared/history/prebuild_sections.tsv` (5 historical runs per gate `a/b/c`; `gate_d` has no history rows).

Historical `fail_rate / mean_cost` per contract clause AS-013:

- `gate_a_reliable` : 0/5 `fail_rate`=0.0, `mean_cost`=800ms → `ratio`=0.0
- `gate_b_moderate` : 1/5 `fail_rate`=0.2, `mean_cost`=500ms → `ratio`=0.0004
- `gate_c_planted_fail` : 4/5 `fail_rate`=0.8, `mean_cost`=100ms → `ratio`=**0.008** (highest)
- `gate_d_new` : no history → median-rate default (DEC-10)

Invoking `gate_runner.sh --order history-cost` MUST execute/report `gate_c_planted_fail` **FIRST** despite it being declared last — this is the literal task-text claim: "a planted failing gate placed last in section order is reported first" (tasks.md T054; plan.md T-C04; contract AS-013).

`expected.json` asserts: `execution_order[0] == "gate_c_planted_fail"` and its verdict is `FAIL` (the gate genuinely fails on every real invocation, never a stub).

Design note (documented inference, no contract field names this explicitly):

the contract's `verdicts/v1` output schema fixes `results` as sorted-by- `gate_id` for determinism (C-002/C-003), so "reported first" cannot be observed from `results` 'order alone. This fixture's driver therefore expects the real tool to additionally emit an `execution_order` array (the actual run sequence) alongside the canonical `results` array — the only way a deterministic, canonically-sorted output format can ALSO expose which gate the ordering

mechanism genuinely ran first. This is the RED test's own design assumption for the field name; the real implementation may name it differently, in which case this fixture's driver assertion (not the fixture data itself) is the single place to update.